

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_6c18d403-385\agent-abe065104e9bb784f.jsonl`  
- SHA-256: `2f7ae19b693d7c29a4532d4dcd47923139a08b6e780cc3a751d43e1d3efd9306`  
- Source modified: `2026-07-06T20:59:18+00:00`  
- Imported at: `2026-07-08T16:00:06+00:00`  
- project: `wf_6c18d403-385`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T20:56:00.636Z)

Reviewing GitHub PR #50 of the repo at C:/Users/avidu/Projects/Telegram ? "feat(policy): add F1 pipeline primitives" (branch feat/f1-policy-engine -> main).
It is the first F1 slice of ADR 0012 (policy pipeline, just merged as #40). +280/-14, 4 files:
- src/tg_video_filter/policy.py (NEW): PolicyContext, StageResult (+ pass_/fail helpers), PolicyStage Protocol, evaluate_pipeline().
- src/tg_video_filter/db.py: NEW load_policy_config/save_policy_config (raw JSON dict over policies.config_json);
 load_filter_config now uses model_validate(_policy_config_json_to_dict(...)) instead of model_validate_json;
 save_filter_config now MERGES cfg.model_dump(mode='json') into the existing raw config dict (preserving unknown keys) instead of replacing the whole blob with cfg.model_dump_json().
- tests/test_db.py + tests/test_policy.py: primitives tests + a preserve-unknown-keys regression test.

A checked-out copy of the PR is at C:/Users/avidu/Projects/_wt-pr50 (read files there or via 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f1-policy-engine:<path>').
Run repros with: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (build schema via db._apply_schema on aiosqlite ":memory:").

BACKGROUND ? why this PR exists: ADR 0012 review found that load_filter_config/save_filter_config would ERASE future pipeline keys (topics/security_checks/deep_inspect) when a user runs /set, because save re-serialized via FilterConfig (which drops unknown keys). F1 fixes that persistence boundary. The main reviewer already verified the happy path works (pipeline keys survive a /set; flat fields update). Your job: find edge cases / bugs / regressions the happy path missed.

Ground rules:

- READ actual code, cite file:line. Run repros. Authoritative gate ALREADY RUN: ruff clean, 501 passed @ 96.63%.
- Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario per finding. Rank most-severe first.
- Severity: high (real bug/regression/data-loss), medium (edge case, latent, asymmetry), low (typing/hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code, run a repro with PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if

handled / can't occur / pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled.
Corrected severity.
FINDING (persistence): save_filter_config silently drops all pipeline keys on a corrupt config, while load_policy_config raises on the same data (read/write asymmetry)
severity low | src/tg_video_filter/db.py:1775
summary: save_filter_config catches (json.JSONDecodeError, ValueError) and resets raw_config to {} ? silently discarding any unrecoverable pipeline keys ? whereas load_policy_config and load_filter_config do NOT catch and instead raise on the identical corrupt config_json. The two sides of the boundary disagree on how to treat a corrupt blob.
scenario: policies.config_json becomes non-object or malformed (e.g. partial write, manual edit, or a future bug). A /set command then runs: save_filter_config swallows the decode error, sets raw_config={}, and overwrites the policy with only the flat FilterConfig fields ? any topics/security_checks/deep_inspect that might have been salvageable are gone (verified: 'save_filter_config over malformed -> config now: {flat defaults only}'). But if instead the manager restarts first, load_filter_config (manager.py:140) or a future load_policy_config caller raises and the session/command errors out. So the same corrupt state either silently self-heals-with-data-loss or hard-fails depending on which path runs first.
evidence: Repro (r1.py, r3.py): save_filter_config over '{not valid json}' produced a config with only flat FilterConfig keys and no error; load_policy_config/load_filter_config over the same blob raised. db.py:1773-1776 catches and sets raw_config={}; db.py:1310-1320 (load_policy_config) and 1749-1759 (load_filter_config) have no try/except.
rec: Decide one policy for corrupt configs and apply it on both sides ? either both raise (safest: refuse to silently drop) or both degrade to defaults with a logged warning. At minimum, log a warning in save_filter_config's except branch so silent pipeline-key loss is observable.

2. assistant (2026-07-06T20:56:02.974Z)

I'll verify this finding adversarially. Let me read the actual code at the cited lines.

3. user (2026-07-06T20:56:04.498Z)

```
1290
1291
1292     def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1293         """Decode a policy config JSON object.
1294
1295         ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1296         expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
1297         nested stage config can coexist.
1298         """
1299         decoded = json.loads(raw)
1300         if not isinstance(decoded, dict):
1301             raise ValueError("policy config_json must decode to a JSON object")
1302         return decoded
1303
1304
1305     def _policy_config_dict_to_json(config: dict[str, object]) -> str:
1306         """Encode policy config deterministically and compactly."""
1307         return json.dumps(config, separators=(",", ":"), sort_keys=True)
1308
1309
1310     async def load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str,
object]:
1311         """Load the raw default policy config as a JSON object.
1312
1313         Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1314         as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1315         through the flat ``FilterConfig`` model.
1316         """
1317         policy = await get_default_policy(conn, account_id)
1318         if policy is None:
1319             return FilterConfig().model_dump(mode="json")
1320         return _policy_config_json_to_dict(policy.config_json)
1321
1322
```

```

1323 async def save_policy_config(
1324     conn: aiomysqlite.Connection, account_id: int, config: dict[str, object]
1325 ) -> None:
1326     """Persist the raw default policy config object."""
1327     existing = await get_default_policy(conn, account_id)
1328     await upsert_policy(
1329         conn,
1330         Policy(
1331             id=existing.id if existing is not None else None,
1332             account_id=account_id,
1333             name="default",
1334             config_json=_policy_config_dict_to_json(config),
1335             is_default=True,
1336             created_at=(existing.created_at if existing is not None else _now()),
1337             updated_at=_now(),
1338         ),
1339     )
1340
1341
1342 # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1343
1344
1345 async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) ->
Destination:
1346     """Insert or update a destination row by (account, provider, target)."""
1347     await conn.execute(
1348         """INSERT INTO destinations
1349             (account_id, provider, target, kind, title, is_output, created_at)

```

4. user (2026-07-06T20:56:04.913Z)

```

1730         document_id=(
1731             _safe_int_cast(item.document_id, "document_id")
1732             if item.document_id is not None
1733             else None
1734         ),
1735         duration_s=item.duration_s,
1736         width=item.width,
1737         height=item.height,
1738         size_bytes=item.size_bytes,
1739         mime_type=item.mime_type,
1740         matched=item.matched,
1741         forwarded=False,
1742         seen_at=item.seen_at,
1743     )
1744
1745
1746 # --- filter config shims (backwards-compatible view over policies) -----
1747
1748
1749 async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1750     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1751
1752     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1753     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1754     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1755     """
1756     policy = await get_default_policy(conn, user_id)
1757     if policy is None:
1758         return FilterConfig()
1759     return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1760
1761
1762 async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:

```

```

1763     """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1764
1765     ADR 0012: future pipeline keys live in the same ``policies.config_json``
1766     object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1767     merge the flat config fields into the existing object so unknown policy
1768     keys survive legacy ``/set`` command updates.
1769     """
1770     existing = await get_default_policy(conn, user_id)
1771     raw_config: dict[str, object] = {}
1772     if existing is not None:
1773         try:
1774             raw_config = _policy_config_json_to_dict(existing.config_json)
1775         except (json.JSONDecodeError, ValueError):
1776             raw_config = {}
1777     raw_config.update(cfg.model_dump(mode="json"))
1778     await save_policy_config(conn, user_id, raw_config)
1779
1780
1781 # --- bulk helpers (used by the manager on startup) -----
1782
1783
1784 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1785     conn: aiosqlite.Connection, user_id: int, limit: int = 100
1786 ) -> list[VideoRecord]:
1787     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1788     conn.row_factory = aiosqlite.Row
1789     async with conn.execute(
1790         """SELECT ci.*, s.provider_source_id AS src_pid
1791            FROM content_items ci
1792           JOIN sources s ON s.id = ci.source_id
1793          WHERE ci.account_id = ?
1794          ORDER BY ci.id DESC LIMIT ?""",
1795         (user_id, limit),
1796     ) as cur:
1797         rows = await cur.fetchall()
1798     return [
1799         VideoRecord(
1800             id=r["id"],
1801             user_id=user_id,
1802             channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
1803             message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
1804             document_id=(
1805                 _safe_int_cast(r["document_id"], "document_id")
1806                 if r["document_id"] is not None
1807                 else None
1808             ),
1809             duration_s=r["duration_s"],
1810             width=r["width"],
1811             height=r["height"],
1812             size_bytes=r["size_bytes"],
1813             mime_type=r["mime_type"],
1814             matched=bool(r["matched"]),
1815             forwarded=False,
1816             seen_at=r["seen_at"],
1817         )
1818         for r in rows
1819     ]
1820
1821
1822 __all__ = [
1823     "init_db",
1824     "compute_dedup_key",
1825     "_safe_int_cast",
1826     # Account + session repos (ADR 0010)
1827     "upsert_account",

```

```
1828         "get_account",
1829         "get_account_by_telegram_id",
```

5. assistant (2026-07-06T20:56:07.508Z)

Both sides confirmed. Now let me run the repro to verify the described behavior.

6. user (2026-07-06T20:58:16.645Z)

Command running in background with ID: bp1i5xv4q. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bp1i5xv4q.output. You will be notified when it completes. To check interim output, use Read on that file path.

7. user (2026-07-06T20:58:24.865Z)

Traceback (most recent call last):

```
File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r1.py", line 39, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r1.py", line 13, in main
    acc = await db.upsert_account(conn, Account(id=None, telegram_id=42, created_at=db._now()))
    ~~~~~^~~~~~

File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in __init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for Account
id
  Input should be a valid integer [type=int_type, input_value=None, input_type=NoneType]
  For further information visit https://errors.pydantic.dev/2.13/v/int_type
```

8. assistant (2026-07-06T20:58:26.718Z)

Let me check the Account model and upsert_policy signatures.

9. user (2026-07-06T20:58:29.550Z)

```
src/tg_video_filter/models.py:35:class Account(BaseModel):
src/tg_video_filter/models.py:186:class Policy(BaseModel):
---upsert_account---
937:async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
1245:async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1281:async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy | None:
```

10. assistant (2026-07-06T20:58:31.973Z)

Rather than fight the model constructors, let me look at how existing tests set up an account+policy, and reuse that approach.

11. user (2026-07-06T20:58:33.511Z)

```

110:     return await db.upsert_account(conn, Account(id=aid, telegram_id=aid))
114:     created = await db.upsert_account(
134:     await db.upsert_account(conn, Account(id=USER_ID, telegram_id=USER_ID))
167:     await db.upsert_account(conn, Account(id=1, telegram_id=1))
172:     await db.upsert_account(conn, Account(id=2, telegram_id=2))
177:     await db.upsert_account(conn, Account(id=3, telegram_id=3))
187:     await db.upsert_account(conn, Account(id=USER_ID, telegram_id=USER_ID))
460:     await db.save_filter_config(conn, USER_ID, custom)
465:async def test_save_filter_config_overwrites(conn: aiosqlite.Connection) -> None:
467:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=100))
468:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=200))
473:async def test_save_filter_config_preserves_unknown_policy_keys(
490:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_size_mb=512))
507:     await db.save_filter_config(conn, USER_ID, cfg)
521:         await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_mode=mode))
569:         await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_target=target))
762:         # Legacy data preserved in the deprecated tables.
1022:async def test_upsert_source_updates_title_but_preserves_watched_on_conflict(
1047:     assert updated.watched is True # watched preserved, not clobbered
1159:     policy = await db.upsert_policy(
1176:     """save_filter_config ? load_filter_config round-trips through policies."""
1179:     await db.save_filter_config(conn, USER_ID, custom)
1392:     - ALL rows are preserved (no silent drops ? fix #1 & #3).
1445:         # different message/channel. Fix #3 must preserve both.
1489:     ), "fix #3: collision loser not preserved at tg:msg:"

```

12. assistant (2026-07-06T20:58:36.084Z)

`Account(id=aid, telegram\_id=aid)` works. Let me look at the preserve-unknown-keys test to mirror setup.

13. user (2026-07-06T20:58:36.742Z)

```

473     async def test_save_filter_config_preserves_unknown_policy_keys(
474         conn: aiosqlite.Connection,
475     ) -> None:
476         """ADR 0012: legacy config updates must not erase pipeline config keys."""
477         await _seed_user(conn)
478         await db.save_policy_config(
479             conn,
480             USER_ID,
481             {
482                 "min_duration_s": 5,
483                 "max_duration_s": 7200,
484                 "topics": {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"},
485                 "security_checks": {"blocked_domains": ["sketchy.example"]},
486                 "deep_inspect": True,
487             },
488         )
489
490         await db.save_filter_config(conn, USER_ID, FilterConfig(max_size_mb=512))
491
492         raw = await db.load_policy_config(conn, USER_ID)
493         assert raw["max_size_mb"] == 512
494         assert raw["topics"] == {"include": ["AI"], "exclude": ["Crypto"], "mode": "any"}
495         assert raw["security_checks"] == {"blocked_domains": ["sketchy.example"]}
496         assert raw["deep_inspect"] is True
497         loaded = await db.load_filter_config(conn, USER_ID)
498         assert loaded.max_size_mb == 512
499
500
501     async def test_filter_config_with_null_bounds_round_trips(conn: aiosqlite.Connection) ->
None:
502         """None bounds (rule disabled) must survive the JSON round trip."""
503         await _seed_user(conn)

```


RuntimeError: Event loop is closed

16. assistant (2026-07-06T20:58:58.137Z)

The behavior is fully reproduced (the trailing traceback is just aiosqlite connection cleanup noise, unrelated to the finding):

- `load\_policy\_config` RAISES JSONDecodeError on corrupt config
- `load\_filter\_config` RAISES JSONDecodeError on the same data
- `save\_filter\_config` swallows the error, resets to `{}`, and writes back **only flat FilterConfig defaults** ? the `topics` and `deep\_inspect` pipeline keys are permanently gone, no error, no log.

The asymmetry is real and ships in this PR's diff (lines 1770-1778 are new code; the `try/except` swallowing is the new save logic; the load paths at 1310-1320 and 1749-1759 have no equivalent guard).

Now let me assess severity honestly. The finding claims **low**. Let me verify the "no log" claim ? check whether there's any logging in that except branch, and whether corrupt config_json is a realistic state.

17. user (2026-07-06T20:59:00.889Z)

```
"""
existing = await get_default_policy(conn, user_id)
raw_config: dict[str, object] = {}
if existing is not None:
    try:
        raw_config = _policy_config_json_to_dict(existing.config_json)
    except (json.JSONDecodeError, ValueError):
        raw_config = {}
raw_config.update(cfg.model_dump(mode="json"))
await save_policy_config(conn, user_id, raw_config)
```

=== any logger in db.py? ===

```
650:     import logging
652:     _logger = logging.getLogger(__name__)
```

18. assistant (2026-07-06T20:59:08.487Z)

Confirmed: the except branch silently sets `raw\_config = {}` with no logging (the only logger in db.py is at line 650, in an unrelated function).

The finding is CONFIRMED. On severity: the finding self-assigns **low**, but let me weigh it. The scenario is real data-loss (pipeline keys erased) but only triggers on an **already-corrupt** config_json ? which under normal operation (the F1 code always writes valid JSON via `\_policy\_config\_dict\_to\_json`) shouldn't occur. It requires external corruption (partial write, manual edit, future bug). So it's a latent asymmetry / defensive-handling gap, not a bug the happy path or normal operation hits. The read/write disagreement (one raises, one silently self-heals-with-loss) is a genuine consistency defect. **Low is correct** ? possibly arguably medium given the data-loss consequence, but the trigger precondition (pre-corrupted blob) is outside this PR's own write path, so low is defensible and I'll keep it as the finding stated.

19. user (2026-07-06T20:59:18.585Z)

Structured output provided successfully